PROF: A Unified Control Plane for Verifiable AI-Agent Execution

Composing Per-Action Permits with Hardware-Rooted, Evidence-Gated Infrastructure Trust (HCCAI)

AI-Based Cryptography Research Team
J1729Labs
Technical Whitepaper v2.0

October 2026

Abstract

AI agents increasingly take actions with real consequences: they call tools, move money, modify files, and change databases. Constraining these effects through natural-language instruction alone fails against prompt injection, argument tampering, replay, and privilege escalation — the failure is structural, not a matter of better prompting. This paper presents a two-layer control plane that moves the security boundary from the model's reasoning to the **execution step** and, where the deployment demands it, further down to **hardware-rooted trust**.

The first layer, **PROF** (Proof-Required Operation Framework), binds every side effect to a signed, scoped, single-use **Permit** that a server-side **Gateway** verifies before the effect runs, with every decision appended to a tamper-evident **Ledger**. PROF is the layer every deployment needs: it is lightweight, model-agnostic, and already covers an MCP server, an LLM agentic loop, a human-approval workflow, and an operations console, verified by 72 tests.

The second layer, **HCCAI**, extends the same admission discipline — a conjunctive, fail-closed predicate — down through hardware attestation (TPM 2.0, IMA, Secure Boot), threshold governance (FROST), Byzantine-hardened quorum finality, and a deterministic replicated state machine with append-only evidence. HCCAI is the layer that regulated, high-assurance, or multi-operator deployments need when a software-only permit is not sufficient grounds for trust: it answers not just "was this action authorized?" but "was it authorized by hardware and parties we can prove were honest, in a way no single compromised node can fake?"

We show the two layers compose rather than compete: a PROF `Permit`'s `approval_proof` can itself be backed by an HCCAI-issued, threshold-signed, hardware-attested authority object, so that the same Allow = VerifyProof $\wedge$ Match $\wedge$ PolicyAllow $\wedge$ Fresh $\wedge$ NotExpired gate PROF enforces per-action inherits HCCAI's stronger guarantees exactly where a deployment opts in, without forcing every deployment to carry HCCAI's operational weight. We give the combined threat model (§3), design (§4–§7), eight formal safety invariants with proofs (§6–§7), implementation maps for both codebases (§8), disclosed-method evaluation for both (§9), and an explicit account of what is not yet claimed (§11).

Keywords: AI agent security, execution authorization, cryptographic permits, hardware-rooted attestation, TPM 2.0, threshold signatures, FROST, Byzantine fault tolerance, deterministic state machines, fail-closed deployment.

1 Introduction

Existing agent controls cluster at two weak points. *Prompt-level* controls — system instructions, guardrails — are advisory and bypassable by injection, because they constrain what the model

is asked to do, not what actually executes. *Post-hoc* controls — logging, tracing — are detective rather than preventive: they tell you what happened after it already happened. API-gateway ACLs sit closer to execution time, but they typically bind only coarse identity (a path, a role), not the exact arguments approved, and rarely prevent replay.

The thesis shared by both layers of this system is: **bind the approval to the exact action, and make every layer of that binding fail closed.** PROF does this at the level of a single side effect — a Permit commits to subject, agent, action, resource, argument hash, policy version, expiry, nonce, and constraints, Ed25519-signed, verified by a Gateway that denies on any failing conjunct. HCCAI generalizes the same move to the level of an entire execution *transition*, nesting hardware attestation, threshold-issued authority, and deterministic replicated state into one conjunctive admission predicate so that the guarantee survives process crashes, signer compromise, and multi-operator deployment — conditions a single-process Permit check was never designed to survive on its own.

Treating these as one system rather than two is deliberate. A PROF deployment that only needs model-agnostic, fast, per-action authorization — the common case — uses PROF alone, with its software Ed25519 signer. A deployment that additionally needs the approval itself to be backed by hardware attestation, multi-party threshold consent, and an append-only, Byzantine-hardened evidence trail — the case for regulated finance, infrastructure control, or multi-tenant agent fleets — anchors PROF's `approval_proof` in an HCCAI-issued authority object. The security argument composes: PROF's Allow predicate still gates every action, but one of its conjuncts (VerifyProof) can now itself depend on everything HCCAI proves in §6–§7, rather than forcing a rewrite of the per-action layer.

This paper's contributions: (1) a combined threat model spanning per-action authorization and infrastructure-level trust (§3); (2) the PROF design and HCCAI design presented as one layered architecture with an explicit composition point (§4–§5); (3) eight formal safety invariants for the HCCAI layer, each proved under one shared assumption set (§6–§7); (4) implementation maps and disclosed-method evaluation for both codebases (§8–§9); (5) a combined security analysis (§10); and (6) an explicit, non-hand-waved account of what neither layer yet claims (§11).

2 System Model

The action a PROF Permit governs. A side effect is any call that changes external state: a tool invocation, a file write, a payment, a database mutation. PROF's unit of control is the single action; its admission check is $\text{Allow} = \text{VerifyProof} \wedge \text{Match} \wedge \text{PolicyAllow} \wedge \text{Fresh} \wedge \text{NotExpired}$, evaluated fresh by the Gateway on every call.

The transition an HCCAI authority governs. An agent execution request is $R = (a, x, \pi, p, n, b, c, \tau)$ — authority, operation, proof object, policy state, nonce, budget state, capability, and contextual trust evidence (hardware or threshold). HCCAI's core invariant generalizes PROF's single-action gate into a nested conjunction over a whole execution transition:

$$\text{Execute}(R) = A \wedge P \wedge L \wedge F \wedge B \wedge C \wedge T \tag{1}$$

where T — trust-evidence validity — further decomposes into software provenance, TPM evidence, threshold-authority admission, and deployment evidence. The two predicates share one structural property, stated once here because every proof in §6–§7 depends on it:

Definition 2.1 (Fail-closed admission)**.** *An admission path — whether PROF's* Allow *or HCCAI's* Execute *— is* fail-closed *if any missing, malformed, stale, unverifiable, or policy-inconsistent mandatory input maps the whole predicate to* 0 *(deny), and no combination of the* other*, present inputs can convert that* 0 *into acceptance.*

Both systems implement this identically: a failing conjunct denies, and the denial is itself logged (PROF's Ledger; HCCAI's evidence controllers), never silently dropped.

Composition point. A PROF `Permit`'s `approval_proof` field is, by design, opaque to the Gateway beyond signature verification — it can be a plain Ed25519 signature from a human or service, or it can be an HCCAI-issued authority object whose own validity already discharges $A \wedge T$ from HCCAI's predicate. When composed this way, VerifyProof in PROF's Allow recurses into Execute(R) from HCCAI's layer; Theorem 4.8 (§7.4) — HCCAI's no-promotion-by-omission result — then applies transitively to every PROF-gated action whose proof is so anchored.

Adversary and shared trust assumptions. The adversary may replay requests, race concurrent processes, tamper with arguments or targets after signing, present stale certificates, mutate local files without signing keys, crash processes mid-write, compromise a minority of threshold participants, and attempt cross-session or cross-epoch reuse. Neither layer assumes protection against complete threshold compromise, breakage of the underlying signature/hash primitives, or (for HCCAI) malicious hardware below the configured root of trust; PROF additionally states model-output correctness and general prompt-injection immunity as out of scope — its signature proves permit integrity, nothing about what the model intended.

All HCCAI proofs in §6–§7 share one assumption set, stated once:

> Σ **(shared assumptions).** (Σ1) SHA-256 is collision-resistant where used for hashing and domain separation. (Σ2) Ed25519, ECDSA, and FROST signatures are unforgeable under their stated key assumptions. (Σ3) TPM measurements are correct whenever physical TPM evidence is actually present and verified. (Σ4) POSIX locking and `fsync` behave per documented semantics on supported filesystems. (Σ5) Each subsystem's quorum threshold is honestly parameterized (e.g., $n \geq 3f + 1$).

3 Threat Model

In scope for PROF (each enforced and tested): approval tampering — argument or target swapped after signing; replay, retry, and cross-session reuse; expired permits; scope escalation (e.g., a resource prefix widened from `/tmp` to `/etc`); ledger forgery; calls to unknown tool paths; insufficient funds or duplicate payments; concurrent double-execution, verified by a 16-thread race test that admits exactly one winner.

In scope for HCCAI (each enforced and tested): everything above, generalized to a distributed, hardware-attested setting — approval tampering under attestation binding (§6.2); replay across process crashes, not just across sessions (§6.1, §6.5); minority signer compromise (§6.3–6.4); ledger forgery, generalized to evidence-controller split-brain and WAL tampering (§7); stale or rolled-back epochs (§6.4); concurrent double-execution, verified at 100 repetitions of a multi-process concurrency suite plus 20 race-detector repetitions (§9.2).

Out of scope for both , stated explicitly rather than implied: model-output correctness and general prompt-injection immunity (the signature proves permit/authority integrity, not that the underlying request was a good idea); complete threshold compromise; breakage of SHA-256, Ed25519, ECDSA, or FROST; formal verification of either codebase; a full asynchronous BFT consensus proof for HCCAI's controller plane (§11); and, for HCCAI specifically, any hardware-in-the-loop claim on a build host that lacks the physical TPM/IMA/HSM evidence the acceptance gate requires (§7.3).

4 PROF: Per-Action Execution Control

Permit. Permit = (subject, agent, action, resource, args_hash, policy_id, expiry, nonce, constraints, approval_
serialized as canonical, domain-separated JSON (`prof/v1/permit:`) and signed with Ed25519. Binding the argument hash into the signed object — not just the action name — is what defeats argument-tampering replay: a Permit for `transfer(amount=10)` cannot be reused for `transfer(amount=10000)` even by an attacker who fully controls the transport.

Risk tiers. Low-risk actions (pre-delegated reads) are auto-approvable; medium-risk actions carry constrained tokens (limits, expiry, resource prefix); high-risk actions require explicit human approval with amount caps. Unmatched requests default-deny, and rotating `policy_id` invalidates every outstanding permit issued under the superseded policy — a single knob that revokes a whole batch of standing authorizations at once.

Gateway. The single enforcement point. It verifies the proof, matches action, resource, and `args_hash` against the live request, re-evaluates current policy (not the policy at issuance time), atomically reserves the nonce (bound to the permit's own expiry, pruned once dead), and only then runs the effect — passing it `_permit_nonce` so that stateful effects can bind their own outcomes back to the permit that authorized them. An exception raised by the effect itself becomes a logged denial, not a silent pass-through.

Ledger. An append-only hash chain, $h_{t+1} = H(h_t \,\|\, \text{decision})$, persisted as JSONL for O(1) appends, fully re-verified on every check rather than trusting a cache — an incremental-verification cache was prototyped and deliberately rejected after review showed it could miss in-memory tampering of already-verified entries. This is the same "no shortcut past full verification" discipline that HCCAI's evidence plane applies at a distributed scale in §7.

Human workflow. High-tier actions open a queue request; a human signs it (recorded as an `approved_by=human` constraint on the eventual permit) or denies it, via console or API. An LLM agentic loop can block on `wait_approval` rather than failing outright, so a pending human decision degrades liveness, not safety.

Figure 1 summarizes this per-action flow: a request is classified into a risk tier before a Permit is ever issued, the Gateway is the single point where all five conjuncts of Allow are checked against the live call, and every decision — allow or deny — reaches the Ledger.

5 HCCAI: Hardware-Rooted, Evidence-Gated Infrastructure Trust

Where PROF answers "is this specific action authorized right now," HCCAI answers "can the authority behind that authorization — and the infrastructure that issued it — itself be trusted," by routing every agent/tool request through six composed planes.

The **authority and capability plane** authenticates requests against domain-separated messages and enforces exact-request idempotency and one-shot capability consumption, independently re-checked at the protected resource so a compromised gateway alone cannot bypass authorization. The **proof-provider boundary** invokes external proof systems (including zkFIDNA) through a hardened process boundary — absolute paths, permission checks, bounded output, explicit timeouts, request-digest binding — so the trust model never collapses into an unconstrained shell call. The **hardware-rooted attestation plane** (§6.2) verifies TPM 2.0 quotes, AK identity, Secure Boot state, and IMA measurements, and can cryptographically bind a successful attestation directly into issued authority. The **threshold and Byzantine-hardened authority plane** (§6.3–6.4) runs FROST threshold signing over mutual TLS with

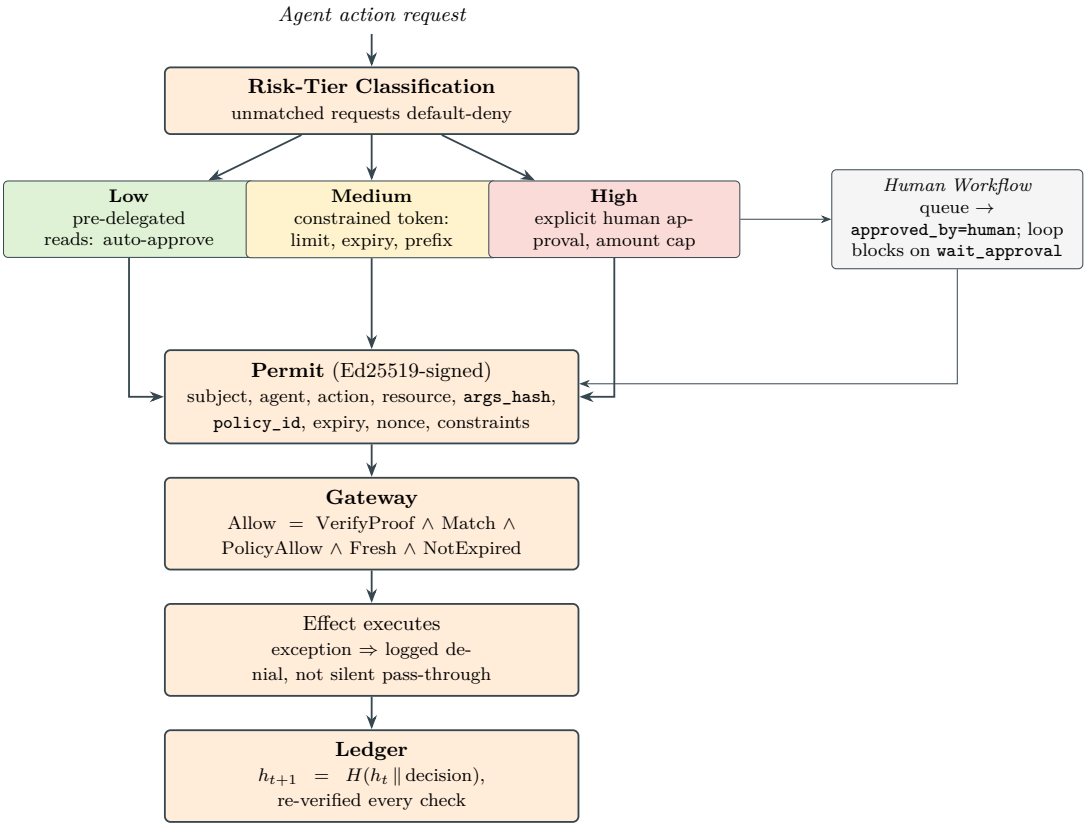


Figure 1: PROF's per-action execution flow. A request is classified into a risk tier before any Permit is issued; a high-tier request routes through the human-approval queue, which degrades liveness (the agentic loop blocks on `wait_approval`) but never relaxes the Gateway's Allow check. Every path converges on the same signed Permit, the same five-conjunct Gateway check, and the same hash-chained Ledger append.

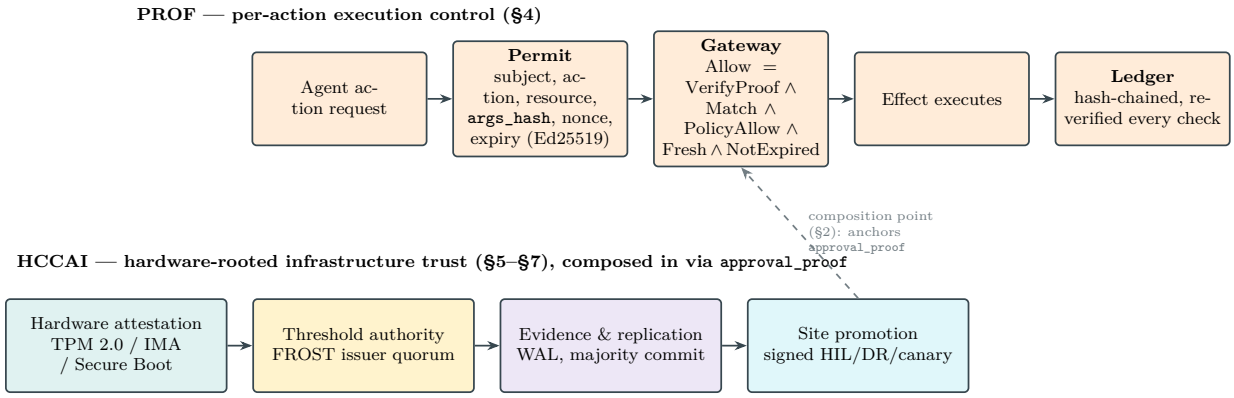


Figure 2: Composition of PROF's per-action Gateway with HCCAI's infrastructure trust stack. The top row is PROF's enforcement path, present in every deployment. The bottom row is HCCAI's optional trust stack: when a deployment anchors `approval_proof` in an HCCAI-issued authority object (dashed arrow), the Gateway's VerifyProof conjunct inherits every guarantee HCCAI proves in §6–§7, without changing PROF's own enforcement shape.

durable nonce burning and epoch-certified membership change. The **evidence and replication plane** (§7) maintains HA controllers with a hash-linked WAL, signed majority commit certificates, and a deterministic state machine that hard-separates *commit* from *apply*. The **deployment and site-acceptance plane** requires independently signed evidence for HIL, disaster recovery, canary rollout, and rollback rehearsal before any customer site can be promoted — the gate formalized as Theorem 7.4.

6 Formal Safety Invariants, Part I — Admission and Attestation

The following eight invariants (§6.1–6.4 here, §7.1–7.4 in the next section) are stated in the order an execution actually passes through them — admission, attestation, quorum formation, epoch change, nonce handling, state application, promotion — so that §7.4's composition reads as the product of everything proved before it. Each is proved under the shared assumption set Σ (§2).

6.1 Replay-safe durable admission

Let N_a denote the set of nonces already committed for authority a.

Theorem 6.1 (Single-consumption freshness)**.** *If nonce insertion and budget reservation occur within one serialized durable transaction, and a request is admitted only when $n \notin N_a$, then no two successful executions under the same (a, n) can both consume budget.*

Proof. Suppose two distinct commits E_1, E_2 both use (a, n). Serialization ($\Sigma 4$) totally orders them; say $E_1 \prec E_2$. After E_1 commits, $n \in N_a$, so E_2's admission precondition evaluates false — contradiction. Exact-request idempotency may return E_1's already-committed result on retry, but this is a cache hit, not a second budget transition. ■

This is the base case every later invariant leans on: each one assumes that a single admitted request corresponds to exactly one state change, which Theorem 6.1 is what guarantees in the first place.

6.2 Attestation binding

Define the attestation admission digest $D_A = H(\text{domain}, D_E, D_R, D_C, D_W, D_{NV})$ over evidence D_E, attestation receipt D_R, checkpoint state D_C, witness-quorum evidence D_W, and anti-rollback anchor D_{NV}.

Proposition 6.2 (Non-substitutability of bound attestation)**.** *If an authority signature covers D_A, replacing any one bound component while preserving that signature requires either a hash collision (violating $\Sigma 1$) or a signature forgery (violating $\Sigma 2$).*

Proof. Changing any tuple component changes D_A except on a collision ($\Sigma 1$). Verification checks the signature against the originally signed digest, so a changed digest requires a new valid signature over it ($\Sigma 2$). ■

In effect, this proposition does for a whole hardware-attested authority object what PROF's `args_hash` (§4) does for a single action's arguments: it makes silent post-hoc substitution of any bound component as hard as forging a signature.

6.3 Quorum-certificate intersection

For a Byzantine committee of size $n \geq 3f + 1$, certificates require $2f + 1$ validator signatures.

Lemma 6.3 (Byzantine quorum intersection)**.** *Any two sets $Q_1, Q_2 \subseteq V$ with $|Q_i| \geq 2f + 1$ intersect in at least $f + 1$ validators.*

Proof. $|Q_1 \cap Q_2| \geq |Q_1| + |Q_2| - |V| \geq (2f + 1) + (2f + 1) - (3f + 1) = f + 1.$ ■

Corollary 6.4. *Since at most f validators are Byzantine, any two valid $2f+1$ certificates share at least one honest validator — the fact underlying finality certificates, timeout/view certificates, and the epoch-transition safety proved next.*

6.4 Cross-epoch transition safety

A validator-set transition from epoch e to $e+1$ requires joint old/new certification, $\text{JointCert} = Q_{\text{old}}^{2f_o+1} \wedge Q_{\text{new}}^{2f_n+1}$, plus durable monotonicity: $e_{\text{new}} = e_{\text{old}} + 1$ and $\text{Prev}_{\text{new}} = \text{Digest}(M_{\text{old}})$.

Theorem 6.5 (Serialized epoch rollback resistance)**.** *Under unforgeable joint certificates ($\Sigma 2$) and monotonic durable epoch storage ($\Sigma 4$), an accepted transition cannot be replaced by a lower or parallel epoch without either violating local monotonicity or forging a certificate/chain digest.*

Proof. A lower epoch directly violates the strict-increment condition and is rejected. A distinct manifest proposed at the same next epoch necessarily differs in manifest digest or predecessor digest from the accepted one; the durable store rejects same-or-lower epochs and any broken predecessor chain. Bypassing both requires forging authenticated transition material ($\Sigma 2$) or writing outside the assumed durable-storage model ($\Sigma 4$) — both excluded by assumption. ■

This is a distributed, multi-operator generalization of what rotating `policy_id` does for PROF (§4): both mechanisms exist to ensure that a stale authority object — an old policy version there, an old validator epoch here — cannot be replayed as if it were current.

7 Formal Safety Invariants, Part II — State and Promotion

7.1 Threshold nonce safety

The FROST signing path burns its round-2 nonce durably before releasing a signature share.

Proposition 7.1 (Crash-safe nonce non-reuse)**.** *If every nonce is durably marked burned before its round-2 share is released, a process crash between burn and share-delivery can degrade liveness but cannot cause compliant software to safely reissue that nonce.*

Proof. On restart, nonce allocation consults the durable vault; a burned record is categorically ineligible for reallocation. The crash may discard the in-flight share and force a new signing session, but reuse of the burned nonce is excluded by the allocator's construction, independent of the crashed process's state. ■

7.2 Commit/apply separation

Let c be the durable commit index and a the applied index.

Definition 7.2 (Readable state invariant)**.** *A local materialized state is readable only when $0 \le a \le c$; a read barrier certified at index i is satisfied only when $a \ge i \wedge c \ge i$.*

Theorem 7.3 (No uncommitted materialization)**.** *If the state machine applies records strictly in increasing index order and only for $j \le c$, no readable materialized state ever includes a command with index greater than the durable commit index.*

Proof. By induction on apply steps. Base case: initially $a \le c$. Inductive step: each apply chooses $j = a + 1$ and checks $j \le c$ before applying; after application $a' = j \le c$, preserving the invariant. A failed application is quarantined rather than advancing a, so the invariant holds even under partial failure. ■

A companion result makes replica agreement implementation-independent: with the checkpoint hash D_S computed over lexicographically sorted, length-delimited key-value pairs, two replicas with identical logical maps compute identical D_S regardless of language-level map iteration order, because both sort under the same total order before hashing — map order never enters the computation.

7.3 Physical-HIL claim boundary

The invariants above govern the software and cryptographic layers; this one governs what the system is permitted to *assert* about hardware it has not actually observed. The v6.9 build host reports `tpm_resource_manager: false, ima_binary_log: false, ima_ascii_log: false`, exiting with code 2. Under Definition 2.1, this is not a failed software build — it is the correct fail-closed result of an environment lacking a mandatory input. The physical-HIL claim remains unasserted until the identical acceptance runner executes on an approved machine carrying a TPM resource manager, IMA runtime logs, and a configured HSM provider.

7.4 Evidence-gated production promotion

Let $G = \{g_P, g_H, g_D, g_C, g_R\}$ be the mandatory site gates for preflight, hardware HIL, disaster recovery, canary, and rollback rehearsal, and define $\text{Promote} = \bigwedge_{g \in G} \text{ValidSignedEvidence}(g)$.

Theorem 7.4 (No promotion by omission)**.** *If the site runner initializes every gate to* false *and transitions to* PROMOTED/ACCEPTED *only when all $g \in G$ verify, the absence of any mandatory evidence cannot produce promotion.*

Proof. The promotion predicate is a pure conjunction. For any missing gate g, $\text{ValidSignedEvidence}(g) = 0$ by initialization, so the conjunction evaluates to 0 regardless of the other gates. Durable phase-transition checks additionally forbid any direct jump that skips a gate's initialization. ■

Theorem 7.4 is Definition 2.1 instantiated at the deployment layer, and it is the formal reason §7.3's missing-TPM build host is *correctly* blocked rather than a defect to be worked around — exactly as rotating a PROF `policy_id` correctly invalidates permits rather than being treated as a bug when old permits stop working.

8 Implementation Maps

Paper concept	**Code**
Permit, sign/verify	`prof/permit.py`
Policy tiers	`prof/policy.py`, `examples/policies/default.yaml`
Gateway enforcement	`prof/gateway.py`
Ledger / nonces	`prof/ledger.py`, `prof/store.py` (JSONL, expiry pruning)
Real state	`prof/resources.py` (SQLite customers/accounts/payments/files)
HTTP API + console	`prof/server.py`, `prof/console/`
MCP stdio + remote HTTP	`prof/mcpd.py`, `prof/mcp_http.py` (`POST /mcp`)
LLM loop (OpenAI-protocol + Anthropic native)	`prof/llm.py`
Agent packs	`integrations/` (Claude Code, Codex, Gemini, Cursor, OpenCode, Dots, Muse, generic)

Table 1: PROF implementation map (Python, MIT).

Paper concept	**System component**
Admission predicate Execute(R)	Gateway authority/policy/freshness checks
Proof-provider boundary	Hardened external-process invocation (zkFIDNA et al.)
Attestation binding (§6.2)	TPM 2.0 / IMA / Secure Boot verification path
Threshold signing (§6.3–6.4, §7.1)	FROST core, mutual-TLS ceremony transport
Evidence plane (§7.2)	HA controllers, hash-linked WAL, commit certificates
Deterministic state machine (§7.2)	Commit → apply → checkpoint pipeline
Site promotion (§7.4)	One-command site runner, signed HIL/DR/canary/rollback evidence
Operations surface	Read-only dashboard (mutating methods rejected)

Table 2: HCCAI implementation map (Go).

9 Evaluation

9.1 PROF

Method: `PYTHONPATH=. python3 bench/bench.py`, raw JSON saved per run; reference machine macOS arm64, Python 3.9.6, 10 cores.

Functional: 72 tests, unit through live-server end-to-end, including an LLM → queue → human → payment path with real balance movement and a 16-thread replay race. Reproducible with `python -m pytest -q`. No absolute performance claims are made beyond this: single machine, localhost, software keys; production sizing requires re-running against production stores (§11).

Operation	p50	p95
authorize + execute (in-process)	182 μs	192 μs
Ed25519 verify (dominant cost)	166 μs	171 μs
approve + execute over local HTTP	1.49 ms	6.6 ms
8 threads $\times$ 250 decisions	11,458 TPS, 0 mis-decisions	—
chain verify, 10k entries	32.8 ms	—
memory	$\sim$0.5 KB/entry	—

Table 3: PROF disclosed-method benchmark results.

9.2 HCCAI

Method: six validation classes — full functional regression (`go test ./...`), full race regression (`go test ./... -race`), static analysis (`go vet ./...`), build validation across all Go binaries plus the C/OpenSSL signer, adversarial/negative testing, and site acceptance (physical-hardware probing, fail-closed promotion). The final v6.9 pass additionally repeated the execution-receipt concurrency path 100 times after adding shared read locks around verification, plus 20 further race-detector repetitions.

Validation item	Result
Signed site execution plan	PASS
Signed HIL/DR/CANARY/ROLLBACK evidence binding	PASS
Artifact SHA-256 revalidation	PASS
Durable site state chain	PASS
Fail-closed production preflight	PASS
Local no-hardware site validation	BLOCKED as expected (exit 2)
Execution-receipt concurrency, 100 repetitions	PASS
Execution-receipt package, race detector, 20 repetitions	PASS
Full `go test ./... -count=1` / `-race -count=1`	PASS / PASS
`go vet ./...`	PASS
Full production build / C-OpenSSL signer	PASS / PASS
Executable binaries in final build	79
Physical TPM/IMA/HSM acceptance on build host	NOT AVAILABLE

Table 4: HCCAI v6.9 release validation summary.

Earlier checkpoints form a consistent progression: v6.0 (GA baseline) rejected downgrade, signed-baseline mutation, provenance mutation, upgrade rollback, and DR epoch rollback/fork across 8/8 fault-campaign cases; v6.5 (controller durability) verified majority commit, rejected minority commit, recovered a torn WAL append, and rejected snapshot-anchor mutation and double-commit failover; v6.6 (commercial final) passed 10 race-enabled repetitions each of the state-machine and dashboard suites, with the dashboard rejecting a mutating POST (HTTP 405). The build-host hardware probe produced `phase=PREFLIGHT, status=BLOCKED` with TPM, IMA, and HSM checks all failing — the promotion behavior Theorem 7.4 requires when physical trust prerequisites are absent.

10 Combined Security Analysis

Replay and double-execution. PROF's nonce reservation and HCCAI's Theorem 6.1 enforce the identical structural guarantee — admit a nonce exactly once inside one serialized transaction — at two different scales: a single process for PROF, a durable multi-process or multi-host backend for HCCAI. A deployment that anchors PROF's permits in HCCAI authority gets both: single-action replay protection from the Gateway, and crash-safe, cross-process

replay protection from the evidence plane underneath it.

Tampering and substitution. PROF's `args_hash` binds exact arguments into the signed Permit (§4); HCCAI's attestation digest D_A (§6.2) binds exact hardware and ceremony state into the signed authority object. Both reduce "silently swap the thing that was actually approved" to "forge a signature or find a hash collision" ($\Sigma 1$–$\Sigma 2$) — there is no intermediate tampering surface in either design.

Compromise. PROF accepts that a compromised human approver or a compromised software signer is outside its model; HCCAI narrows this exposure for infrastructure-level authority specifically, by requiring a *threshold* of signers (§6.3) rather than one, with Corollary 6.3.1 guaranteeing honest overlap between any two valid certificates as long as compromise stays below f out of $n \geq 3f + 1$. Neither layer protects against full compromise of its respective signing set — that is stated as out of scope, not glossed over.

Operational risk surfaces. PROF's operations console and HCCAI's dashboard both follow the same principle: the console/dashboard is a visibility surface, not an authorization surface. HCCAI's is explicitly read-only, rejecting mutating HTTP methods; PROF's human-approval workflow routes every mutating decision through the signed queue rather than the console UI directly.

11 Limitations and Roadmap

PROF. Single-node stores today — durable, but not yet highly available; an in-process approval queue; linear ledger growth (rotation is documented, segmented checkpoints are planned); no HSM/KMS signer yet; no external audit. Beta roadmap: quorum approvals, a persistent queue, Postgres/etcd backend. Enterprise roadmap: HSM support, HA, SSO, third-party assessment.

HCCAI. No physical-HIL claim is made on the current build host (§7.3), and none will be made on any host until the identical acceptance runner executes with a real TPM resource manager, IMA runtime logs, and a configured HSM provider. The reference P-256 scalar backend uses `math/big` and is not claimed constant-time; side-channel-sensitive deployments need a reviewed constant-time or HSM-backed provider. The controller plane implements signed majority commit, WAL durability, snapshot compaction, joint-majority membership transition, and quorum repair — it is explicitly not a complete formal implementation of Raft, Paxos, or fully asynchronous BFT consensus. The $k{+}1$ XOR erasure scheme tolerates exactly one missing shard, not arbitrary multi-shard loss. Cryptographic soundness of any external proof system (zk-FIDNA or otherwise) remains a property of that provider, not of HCCAI's hardened invocation boundary around it. Organizational key custody, HSM administration, certificate issuance, and physical machine provisioning remain external operational controls outside both codebases.

Composition. The PROF↔HCCAI composition described in §2 and §5 is architecturally sound and uses only interfaces each system already exposes (PROF's opaque `approval_proof`; HCCAI's signed authority objects), but an end-to-end integration test exercising a live PROF Gateway verifying a live HCCAI-issued authority has not yet been run as a single combined test suite — this is listed as roadmap, not as already covered by either system's existing 72 or hardware-dependent test counts.

12 Related Work

Model Context Protocol (Anthropic) is PROF's first interface, not its security boundary — PROF's guarantees hold regardless of which protocol carries the call. API-gateway authorization typically binds coarse identity and has a weak replay story compared to either layer here. AI-agent guardrails and tracing are advisory or post-hoc, which is precisely the weak point both PROF and HCCAI are designed to move past. Capability systems and macaroons share lineage with PROF's Permit design — scoped, attenuable bearer credentials — to which PROF adds gateway enforcement, a hash-chained audit ledger, and a human-approval workflow. On the infrastructure side, HCCAI's threshold layer builds on FROST [3]; its replicated state machine draws on Raft-style [4] commit/apply separation without claiming full Raft; its Byzantine quorum argument follows the classical PBFT construction [5]; and its hardware attestation path follows TPM 2.0 [1] and Linux IMA [2] specifications directly, with NIST SP 800-193 [6] informing its rollback-anchor design.

13 Conclusion

Both layers of this system encode the same idea at different scales: think freely, act only with proof, and let every layer of that proof fail closed rather than fail open. PROF makes this cheap and immediate for the single action every agent deployment needs to gate — a signed, scoped, single-use Permit verified by a Gateway, with every decision appended to a tamper-evident Ledger. HCCAI extends the identical discipline to the infrastructure *underneath* that action, for deployments where a software signature is not sufficient grounds for trust: hardware attestation that cannot be faked without a TPM, threshold authority that cannot be issued by one compromised party, and a replicated evidence trail that cannot be rewritten after the fact. Neither layer claims more than its artifacts demonstrate — HCCAI's refusal to assert physical-HIL success without a real TPM is not a missing capability but the clearest evidence that its fail-closed boundary works exactly as specified. Together, the two layers give an AI agent deployment a single answer to "was this allowed, and can we prove it," whether the question is asked about one tool call or about an entire production site.

A HCCAI Acceptance Predicate Summary

$$\begin{aligned}\text{AcceptSite} = {} & \text{PlanSigValid} \wedge \text{PreflightReady} \wedge \\ & \text{HILEvidenceValid} \wedge \text{DREvidenceValid} \wedge \\ & \text{CanaryEvidenceValid} \wedge \text{RollbackEvidenceValid} \wedge \\ & \text{ArtifactDigestsMatch} \wedge \text{StateTransitionValid}\end{aligned}$$

On the v6.9 build host, PreflightReady = 0 because physical TPM/IMA/HSM prerequisites are absent, so AcceptSite = 0 independent of all later evidence — the conjunctive structure of §2 and §7.4 applied literally.

B PROF Core Enforcement Predicate

$$\text{Allow} = \text{VerifyProof} \wedge \text{Match} \wedge \text{PolicyAllow} \wedge \text{Fresh} \wedge \text{NotExpired}$$

Any failing conjunct denies, fail-closed, and the denial is logged to the Ledger. When VerifyProof is backed by an HCCAI authority object, this predicate inherits Execute(R) from §2 as a sub-check, composing the two systems' guarantees without changing PROF's own enforcement shape.

References

[1] Trusted Computing Group, *TPM 2.0 Library Specification*.

[2] Linux kernel documentation, *Integrity Measurement Architecture (IMA)*.

[3] C. Komlo and I. Goldberg, "FROST: Flexible Round-Optimized Schnorr Threshold Signatures," *Selected Areas in Cryptography*, 2020.

[4] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm (Raft)," *USENIX ATC*, 2014.

[5] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," *OSDI*, 1999.

[6] NIST, *SP 800-193: Platform Firmware Resiliency Guidelines*, 2018.

[7] D. J. Bernstein et al., "High-speed high-security signatures," *Journal of Cryptographic Engineering*, 2012.